

Yenepoya(Deemed to be University)

Project – Low Level Design

on

Zero-Day Scanner for E-commerce Web Portals

Student Name: Punnya K P

Industry Mentor:
Shashank Koram

Guided by: Suzana Sharol
Prabhakar

Table of Contents

1. Introduction.....	6
1.1. Scope of the document.....	6
1.2. Intended Audience.....	6
1.3. System overview.....	6
2. System Design.....	6
2.1. Application Design.....	6
2.2. Process Flow.....	6
2.3. Information Flow.....	7
2.4. Components Design.....	8
2.5. Key Design Considerations.....	8
2.6. API Catalogue.....	8
3. Data Design.....	8
3.1. Data Model.....	8
3.2. Data Access Mechanism.....	9
3.3. Data Retention Policies.....	9
3.4. Data Migration.....	9
4. Interfaces.....	9
5. State and Session Management.....	10
6. Caching.....	10
7. Non-Functional Requirements.....	10
7.1. Security Aspects.....	10
7.2. Performance Aspects.....	10
8. References.....	11

1. INTRODUCTION

1.1 Scope of the Document

This Low-Level Design (LLD) document provides a detailed technical description of the project titled:

“Zero-Day Scanner for E-commerce Web Portals”

The purpose of this document is to explain the internal architecture, module-level implementation, and data handling mechanisms of the system.

This document covers:

- Component-level design of scanning modules
- Multi-threaded crawling and analysis logic
- API structure and communication flow
- Data structures and in-memory storage
- Frontend-backend interaction using SSE
- Security and performance strategies

The system simulates real-world web security reconnaissance and automated testing focusing on:

- Broken links and error pages
- Redirect chains and loops
- Soft-404 detection
- Subdomain discovery
- Form and asset extraction

1.2 Intended Audience

1. Students learning cybersecurity and web scanning concepts
2. Faculty evaluating technical depth and implementation
3. Ethical hacking beginners exploring automation tools
4. Developers interested in crawling and HTTP analysis systems

1.3 System Overview

The system is a web-based and CLI-based scanning tool that analyzes e-commerce websites.

The user enters a target URL, and the system performs:

- Multi-level crawling of links
- Subdomain enumeration
- HTTP status verification
- Soft-404 detection
- Form and asset extraction

The system provides:

- Live scanning logs (via SSE)
- Categorized results (200, 404, redirects, soft-404)
- Export options (JSON, CSV)

2. SYSTEM DESIGN

2.1 Application Design

The Zero-Day Scanner follows an asynchronous client-server architecture.

Unlike simple synchronous systems, this project uses:

- Background scanning threads
- Real-time updates using Server-Sent Events (SSE)
- Concurrent request handling

Architecture Layers:

1. Presentation Layer

- HTML, CSS, JavaScript
- Displays terminal UI and results

2. Application Layer

- Flask handles routing and APIs

3. Processing Layer

- Core scanning engine (crawler, analyzer)

4. Network Layer

- Requests for HTTP calls

5. Concurrency Layer

- ThreadPoolExecutor for parallel scanning

2.2 Process Flow

1. User enters URL and crawl depth
2. Frontend sends POST request to /api/scan
3. Backend generates unique scan_id
4. Background thread starts scanning
5. Subdomain enumeration begins
6. HTML pages are fetched and parsed
7. Links and forms are extracted
8. Links are verified concurrently
9. Soft-404 detection is applied
10. Logs are streamed via SSE
11. Final result is stored in memory
12. Frontend fetches results using /api/result

2.3 Information Flow

- Input: User URL + depth
- Processing:
 - DNS resolution
 - HTML parsing

- HTTP status analysis
- Output:
 - Categorized links
 - Subdomains
 - Forms
 - Summary statistics

Data flows through:

- Queue (for live updates)
- In-memory dictionary (for results)

2.4 Component Design

The system consists of the following components:

1. Scan Controller
 - Handles API requests
 - Creates scan threads
2. Subdomain Enumerator
 - Uses wordlist-based DNS resolution
 - Identifies hidden infrastructure
3. Link Crawler
 - Extracts links from HTML
 - Maintains visited set
 - Supports depth-based crawling
4. HTML Parser
 - Extracts:
 - Links (<a>)
 - Forms (<form>)
 - Assets (img, script, iframe)
5. Status Analyzer

- Checks HTTP status codes
 - Detects redirects
 - Identifies soft-404 pages
6. Concurrency Engine
- Uses multithreading
 - Improves performance
7. Export Module
- Converts results to JSON/CSV

2.5 Key Design Considerations

- Crawl depth limited to 3 (prevents overload)
- Max threads: 20–30 (balanced performance)
- Timeout: 6–10 seconds per request
- Redirect limit enforced
- SSL verification disabled for flexibility
- Duplicate links removed using sets
- User-Agent header used to avoid blocking

2.6 API Catalogue

API	Method	Description
/scan	POST	Start scan
/stream	GET	Live logs (SSE)
/result	GET	Fetch results

3. DATA DESIGN

3.1 Data Model

Scan Object

```
{  
  "target": "example.com",  
  "scan_time": "timestamp",  
  "links": [],  
  "subdomains": [],  
  "forms": [],  
  "summary": {}  
}
```

3.2 Data Access Mechanism

- Data stored in-memory using Python dictionary
- Queue used for streaming logs
- Lists & sets used for processing

No database is used → faster execution

3.3 Data Retention Policies

- Data exists only during runtime
- Cleared after scan completion
- No logs stored permanently
- No cookies/session storage used

3.4 Data Migration

- Not applicable (no database used)
- Future scope:
 - Add Redis / database
 - Store scan history

4. INTERFACES

The system provides:

User Interface

- Input box for URL
- Crawl depth selection
- Live terminal panel
- Results dashboard

External Interfaces

- HTTP/HTTPS communication with websites
- DNS resolution using socket

5. STATE AND SESSION MANAGEMENT

- Fully stateless system
- Each scan is independent
- No session or cookies used
- Data stored temporarily in memory

6. CACHING

- No caching implemented
- Every scan fetches fresh data
- Ensures real-time accuracy

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Security Aspects

- Prevents excessive requests (rate limiting)
- No user data stored
- Input validation for URLs
- Prevents infinite redirects
- Thread-safe operations using locks

7.2 Performance Aspects

- Multithreading improves speed
- Parallel HTTP requests
- Efficient data structures (sets, lists)
- Controlled crawl depth

8. REFERENCES

- Flask Documentation: <https://flask.palletsprojects.com/>
- Requests Library Docs: <https://docs.python-requests.org/en/latest/>